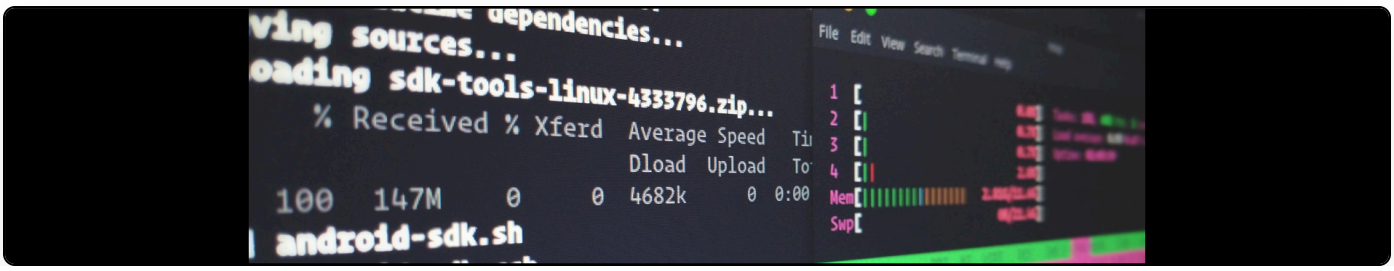
Building a robust, SSL, CRC-Verified server/client solution in the .NET Framework with C#

Cite as: Martin Paul Eve, "Building a robust, SSL, CRC-Verified server/client solution in the .NET Framework with C#", *eve.gd: Martin Paul Eve*, July 23, 2008, https://doi.org/10.59348/3r18y-49604.



Quite a lengthy post here with a lot of code in the hope that my experience of building an integrity-checking SSL (text-only for now) communication system will be of use to somebody else.

The way the system I have designed works is thus:

1. Server sends banner
2. Client sends login
3. Server verifies and then either client or server is free to send commands with sequence numbers

The conditions are that the system must verify every single line of text sent via some kind of CRC (using MD5 here) and disconnect gracefully, raising an event to tell the host application so, if there is a problem.

First of all we need to define some kind of protocol between the client and server. Here's what I came up with for a skeleton.

ProtocolText.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace Shared
{
    /// <summary>
    /// The protocol definition
    /// </summary>
    public static class ProtocolText
    {
        static string _banner = "AUTH";

        /// <summary>
        /// The banner text of the protocol
        /// </summary>
        public static string BANNER
        {
            get { return _banner; }
            set { _banner = value; }
        }

        static string _positive = "YES";

        /// <summary>
        /// The positive response text of the protocol
        /// </summary>
        public static string Positive
        {
            get { return ProtocolText._positive; }
            set { ProtocolText._positive = value; }
        }

        static string _negative = "NO";

        /// <summary>
        /// The negative response text of the protocol
        /// </summary>
        public static string Negative
        {
            get { return ProtocolText._negative; }
            set { ProtocolText._negative = value; }
        }

        static string _LOGINPrefix = "LOGIN";
```

```
        /// <summary>
        /// The prefix to the login command of the protocol
        /// </summary>
        public static string LOGINPrefix
        {
            get { return ProtocolText._LOGINPrefix; }
            set { ProtocolText._LOGINPrefix = value; }
        }

        static string _QUIT = "GOODBYE";

        /// <summary>
        /// The quit command text of the protocol
        /// </summary>
        public static string QUIT
        {
            get { return ProtocolText._QUIT; }
            set { ProtocolText._QUIT = value; }
        }
    }
}
```

Next up, some form of wrapping "commands" inside an API. These are actually just text, but putting them into objects has obvious usability implications.

aCommand.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace Shared.Commands
{
    /// <summary>
    /// The type of this command
    /// </summary>
    public enum CommandType
    {
        BANNER,
        AUTH,
        QUIT,
        PREPAREINDEX
    }

    /// <summary>
    /// The response level from the command
    /// </summary>
    public enum Response
    {
        TERMINAL,
        ERROR,
        WARNING,
        INFORMATION,
        SUCCESS
    }

    /// <summary>
    /// An abstract class for implementing commands
    /// </summary>
    public abstract class aCommand
    {
        long _sequenceNumber = 0;
        string[] _commandText = null;
        bool _hasCommandsToSend = false;
        string _information = string.Empty;
        List<string> _response = new List<string>();

        /// <summary>
        /// When Response.INFORMATION is issued, this property will contain
        /// the information specified in the response
        /// </summary>
        public string Information
        {
```

```
            get { return _information; }
        }

        /// <summary>
        /// The sequence number of this message
        /// </summary>
        public long SequenceNumber
        {
            get
            {
                return _sequenceNumber;
            }
            set
            {
                _sequenceNumber = value;
            }
        }

        /// <summary>
        /// The command text of this message
        /// </summary>
        public string[] CommandText
        {
            get
            {
                return _commandText;
            }
        }

        /// <summary>
        /// Whether there is fresh command text to send
        /// </summary>
        public bool HasCommandsToSend
        {
            get
            {
                return _hasCommandsToSend;
            }
            set
            {
                _hasCommandsToSend = value;
            }
        }

        /// <summary>
        /// The text of the response received so far
        /// </summary>
        public List<string> ResponseText
```

```
        {
            get { return _response; }
        }

        /// <summary>
        /// A method for inheritors to set new command text
        /// </summary>
        /// <param name="commands">The command text to send</param>
        protected void setCommandText(string[] commands)
        {
            _hasCommandsToSend = true;
            _commandText = commands;
        }

        /// <summary>
        /// A method for inheritors to set the Information property
        /// </summary>
        /// <param name="info">The text that the Information property should be set
to</param>
        protected void setInformation(string info)
        {
            _information = info;
        }

        /// <summary>
        /// Append text to the response of this client
        /// </summary>
        /// <param name="msg">The text to append</param>
        public void AddResponse(string msg)
        {
            _response.Add(msg);
        }

        /// <summary>
        /// The type of this command
        /// </summary>
        public abstract CommandType CommandType { get; }

        /// <summary>
        /// Called by aClient whenever a response block is complete
        /// </summary>
        /// <returns>A Response object indicating the status of this
operation</returns>
        public abstract Response ResponseDone();
    }
}
```

Also in our shared library (this is referenced by both the client and the server) we need the CRC checker.

CRC.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Security.Cryptography;
using System.IO;

namespace Shared
{
    /// <summary>
    /// A class to provide CRC functions
    /// </summary>
    public class CRC
    {
        static MD5CryptoServiceProvider cSP = new MD5CryptoServiceProvider();

        /// <summary>
        /// Compute the CRC of a message
        /// </summary>
        /// <param name="message">The message text</param>
        /// <returns>The CRC</returns>
        public static string ComputeCRC(string message)
        {
            return
BitConverter.ToString(cSP.ComputeHash(System.Text.ASCIIEncoding.ASCII.GetBytes(message)
));
        }

        /// <summary>
        /// Compute the CRC of a stream
        /// </summary>
        /// <param name="stream">The stream (probably a file)</param>
        /// <returns>The CRC</returns>
        public static string ComputeCRC(Stream stream)
        {
            return BitConverter.ToString(cSP.ComputeHash(stream));
        }
    }
}
```

Now, the workhorse itself, the actual client. This is responsible for threading, SSL initiation (to some extent), CRC checking and passing message responses to the correct places.

aClient.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Net.Sockets;
using System.Net.Security;
using System.Security.Cryptography.X509Certificates;
using System.Security.Cryptography;
using System.IO;
using System.Threading;
using System.Text.RegularExpressions;
using System.Diagnostics;
using Shared.Commands;

namespace Shared
{
    /// <summary>
    /// A list of client events that inheritors can raise
    /// </summary>
    public enum ClientEvents
    {
        OnAuthFailure,
        OnAuthSuccess
    }

    /// <summary>
    /// An abstract class for implementing SSL, CRC verified communication
    /// between clients
    /// </summary>
    public abstract class aClient
    {
        TcpClient _client = null;
        SslStream _ssl = null;
        StreamReader _reader = null;
        StreamWriter _writer = null;
        bool _shutdown = false;
        bool _isConnecting = false;
        string _server = string.Empty;
        int _sequenceNumber = 0;
        int _connectionAttempts = 0;
        List<aCommand> _commandQueue = new List<aCommand>();
        ManualResetEvent _hasMessages = new ManualResetEvent(false);
        ManualResetEvent _connectingWait = new ManualResetEvent(false);
        Regex msgMatcher = new Regex(@"^(\d+)\s(.+)\sCRC(.+)$");

        public delegate void ClientEvent(aClient client);
        public event ClientEvent OnAuthFailure;
```

```
        public event ClientEvent OnAuthSuccess;
        public event ClientEvent OnShutdown;

        protected State _state = State.Not_Connected;

        public abstract void initSSL(SslStream ssl, string hostname);
        public abstract void connectionInit();
        public abstract void processResponse(Response response, aCommand command);
        public abstract void processNewCommand(string commandText, long
sequenceNumber);

        /// <summary>
        /// States of the client
        /// </summary>
        protected enum State
        {
            Not_Connected,
            Connected,
            Authenticated
        }

        /// <summary>
        /// Creates a new instance of the aClient
        /// </summary>
        /// <param name="client">The underlying TcpClient</param>
        /// <param name="hostname">The hostname</param>
        public aClient(TcpClient client, string hostname)
        {
            _client = client;
            _server = hostname;
        }

        /// <summary>
        /// Instructs the aClient to begin processing messages
        /// </summary>
        /// <remarks>This is a non-blocking operation</remarks>
        public void Start()
        {
            ThreadStart ts = new ThreadStart(messageLoop);
            Thread t = new Thread(ts);
            t.Start();
        }

        /// <summary>
        /// Raise a client event
        /// </summary>
        /// <param name="eventType">The type of event to raise</param>
        protected virtual void onClientEvent(ClientEvents eventType)
```

```
        {
            // Make a temporary copy of the event to avoid possibility of
            // a race condition if the last subscriber unsubscribes
            // immediately after the null check and before the event is raised.
            ClientEvent handler = null;

            switch (eventType)
            {
                case ClientEvents.OnAuthFailure:
                    handler = OnAuthFailure;
                    break;
                case ClientEvents.OnAuthSuccess:
                    handler = OnAuthSuccess;
                    break;
            }

            if (handler != null)
            {
                handler(this);
            }
        }

        /// <summary>
        /// Initialise SSL on the specified client
        /// </summary>
        private void initConnection()
        {
            if (_shutdown) return;

            if (!_isConnecting)
            {
                _isConnecting = true;
                _connectingWait.Reset();

                try
                {
                    _ssl = new SslStream(_client.GetStream(), false, new
RemoteCertificateValidationCallback(ValidateServerCertificate));

                    initSSL(_ssl, _server);

                    _state = State.Connected;

                    _reader = new StreamReader(_ssl);
                    _writer = new StreamWriter(_ssl);

                    connectionInit();
```

```
                _connectionAttempts = 0;
            }
            catch (Exception ex)
            {
                Trace.WriteLine(ex.Message + Environment.NewLine + ex.StackTrace);
                _connectionAttempts++;
            }
            finally
            {
                _connectingWait.Set();
                _isConnecting = false;
            }
        }
        else
        {
            //Block until function is done
            _connectingWait.WaitOne();
        }
    }

    /// <summary>
    /// Handle a connection exception and shut down gracefully
    /// </summary>
    /// <param name="ex">The exception object that triggered this problem</param>
    private void handleException(Exception ex)
    {
        if(ex!=null)
        Trace.WriteLine(ex.Message + Environment.NewLine + ex.StackTrace);

        if (_state == State.Connected)
        {
            //Login was rejected
            if (OnAuthFailure != null) OnAuthFailure(this);

            Shutdown();
        }
        else
        {
            //Connection aborted

            Shutdown();
        }
    }

    /// <summary>
    /// The loop that handles the sending of messages
    /// </summary>
    private void sendLoop()
```

```
        {
            while (!_shutdown)
            {
                //Find commands that have changed
                IEnumerable<aCommand> changedCommands =
_commandQueue.Where(delegate(aCommand tmpCommand)
                { return tmpCommand.HasCommandsToSend == true; });

                foreach (aCommand clientCommand in changedCommands)
                {
                    clientCommand.HasCommandsToSend = false;

                    bool setSequenceNumber = false;

                    //See if it has a sequence number
                    if (clientCommand.SequenceNumber == 0)
                    {
                        setSequenceNumber = true;

                        if (_sequenceNumber == int.MaxValue)
                        {
                            //TODO: Handle int.MaxValue
                        }

                        _sequenceNumber++;

                        clientCommand.SequenceNumber = _sequenceNumber;
                    }

                    try
                    {
                        foreach (string commandLine in clientCommand.CommandText)
                        {
                            string msg = string.Format("{0} {1}",
clientCommand.SequenceNumber, commandLine);
                            string crc = Shared.CRC.ComputeCRC(msg);

                            //Message format:
                            //SEQ_NUM COMMAND ARGS CRCMESSAGE_CRC
                            string msgToSend = string.Format("{0} CRC{1}", msg, crc);

                            _writer.WriteLine(msgToSend);
                            _writer.Flush();
                        }

                        //Send DONE msg
                        string doneMsg = string.Format("{0} {1}",
clientCommand.SequenceNumber, "DONE");
```

```
                        string doneCrc = Shared.CRC.ComputeCRC(doneMsg);

                        string doneMsgToSend = string.Format("{0} CRC{1}", doneMsg,
doneCrc);

                        _writer.WriteLine(doneMsgToSend);
                        _writer.Flush();

                    }
                    catch (Exception ex)
                    {
                        //Decrement sequence number to resend
                        if (setSequenceNumber) _sequenceNumber--;

                        handleException(ex);
                    }
                }

                if (changedCommands.Count() == 0)
                {
                    _hasMessages.WaitOne();
                    _hasMessages.Reset();
                }

            }
        }

        /// <summary>
        /// The main message loop that starts the sending process and
        /// processes incoming messages
        /// </summary>
        private void messageLoop()
        {
            initConnection();

            //Start the send loop in a different thread
            ThreadStart ts = new ThreadStart(sendLoop);
            Thread t = new Thread(ts);
            t.Start();

            //Start the listen loop
            while (!_shutdown)
            {
                string msg = string.Empty;

                try
                {
                    msg = _reader.ReadLine();
```

```
                    if (msg == null) handleException(null);

                    Trace.WriteLine(msg);
                }
                catch (Exception ex)
                {
                    handleException(ex);
                }

                //Check the shutdown flag wasn't set
                if (_shutdown) break;

                //Response format:
                //SEQ_NUM RESPONSE CRCMessageCRC
                //SEQ_NUM RESPONSE CRCMessageCRC
                //SEQ_NUM DONE CRCMessageCRC

                Match msgMatch = msgMatcher.Match(msg);

                if (!msgMatch.Success || msgMatch.Groups.Count != 4)
                {
                    handleCRCException();
                }
                else
                {
                    int commandSeq = 0;

                    if (!int.TryParse(msgMatch.Groups[1].Captures[0].Value, out
commandSeq))
                    {
                        handleCRCException();
                    }
                    else
                    {
                        //Validate the CRC
                        string messageContents = msgMatch.Groups[2].Captures[0].Value;
                        string targetCRC = msgMatch.Groups[3].Captures[0].Value;

                        if (targetCRC != Shared.CRC.ComputeCRC(string.Format("{0} {1}",
commandSeq, messageContents)))
                        {
                            handleCRCException();
                        }

                        //Get the command with this sequence number
                        IEnumerable<aCommand> commands =
_commandQueue.Where(delegate(aCommand tmpCommand)
```

```
                    { return tmpCommand.SequenceNumber == commandSeq; });

                    if (commands.Count() == 0)
                    {
                        //This is an unprecedented communication, not a response
                        processNewCommand(messageContents, commandSeq);
                    }
                    else
                    {
                        aCommand command = commands.ElementAt(0);

                        if (command == null)
                        {
                            handleCRCException();
                        }
                        else
                        {
                            if (messageContents.StartsWith("DONE"))
                            {
                                processResponse(command.ResponseDone(), command);
                            }
                            else
                            {
                                command.AddResponse(messageContents);
                            }
                        }
                    }
                }
            }
        }

        /// <summary>
        /// Handle a CRC exception
        /// </summary>
        private void handleCRCException()
        {
            Trace.WriteLine("CRC error in server response. Resetting connection.");
            handleException(null);
        }

        /// <summary>
        /// Send a command to the remote connection
        /// </summary>
        /// <param name="command">The aCommand to send</param>
        protected void sendCommand(aCommand command)
        {
            _commandQueue.Add(command);
```

```
            _hasMessages.Set();
        }

        /// <summary>
        /// Mark the message queue as having updated data to send
        /// </summary>
        protected void sendCommand()
        {
            _hasMessages.Set();
        }

        /// <summary>
        /// Performs dummy validation of remote certificate. Always returns
        /// true
        /// </summary>
        /// <param name="sender">The calling object</param>
        /// <param name="certificate">The certificate</param>
        /// <param name="chain">The X509 chain object</param>
        /// <param name="sslPolicyErrors">The errors on the certificate</param>
        /// <returns>True</returns>
        private static bool ValidateServerCertificate(
              object sender,
              X509Certificate certificate,
              X509Chain chain,
              SslPolicyErrors sslPolicyErrors)
        {
            return true;
        }

        /// <summary>
        /// Shutdown this aClient gracefully
        /// </summary>
        public void Shutdown()
        {
            _state = State.Not_Connected;
            _reader.Close();
            _writer.Close();
            _ssl.Close();
            _shutdown = true;

            _connectingWait.Set();
            _hasMessages.Set();

            if (OnShutdown != null) OnShutdown(this);
        }
    }
}
```

So, to actually use these classes we need to implement a server object, a client object, some commands and server and client wrappers to instantiate a TcpClient and then pass it to the aClient inheritors.

ServerNetworkClient.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using Shared;
using System.Security.Cryptography.X509Certificates;
using System.Net.Sockets;
using System.Diagnostics;

namespace Server
{
    /// <summary>
    /// A server implementation of aClient
    /// </summary>
    class ServerNetworkClient : aClient
    {
        X509Certificate cert =
X509Certificate.CreateFromCertFile("PUTYOURCERTIFICATEHERE-USE-MAKECERT-TO-
GENERATE.cer");

        /// <summary>
        /// Construct a new server instance around the specified
        /// TcpClient
        /// </summary>
        /// <param name="client">The TcpClient</param>
        public ServerNetworkClient(TcpClient client) : base(client, "") { }

        /// <summary>
        /// Initialise SSL as server
        /// </summary>
        /// <param name="ssl">The SSLStream to authenticate against</param>
        /// <param name="hostname">The current hostname</param>
        public override void initSSL(System.Net.Security.SslStream ssl, string
hostname)
        {
            Trace.TraceInformation("Authenticating as server for SSL.");
            ssl.AuthenticateAsServer(cert, false,
System.Security.Authentication.SslProtocols.Tls, false);
        }

        /// <summary>
        /// Init the connection by sending a banner
        /// </summary>
        public override void connectionInit()
        {
            ServerCommands.AUTHCommand authBanner = new
ServerCommands.AUTHCommand("hello");
```

```
            this.sendCommand(authBanner);
        }

        /// <summary>
        /// Process a new incoming command
        /// </summary>
        /// <param name="messageText">The text of the message</param>
        /// <param name="sequenceNumber">The sequence number of the message</param>
        public override void processNewCommand(string messageText, long sequenceNumber)
        {
        }

        /// <summary>
        /// Process a response from a command
        /// </summary>
        /// <param name="response">The response</param>
        /// <param name="command">The aCommand that raised the response</param>
        public override void processResponse(Shared.Commands.Response response,
Shared.Commands.aCommand command)
        {
            switch (command.CommandType)
            {
                case Shared.Commands.CommandType.BANNER:
                    //AUTH check
                    handleAuth(response, command);
                    break;
            }
        }

        /// <summary>
        /// Handle authentication
        /// </summary>
        /// <param name="response">The response object</param>
        /// <param name="command">The aCommand object</param>
        private void handleAuth(Shared.Commands.Response response,
Shared.Commands.aCommand command)
        {
            switch (response)
            {
                case Shared.Commands.Response.SUCCESS:
                    base.onClientEvent(ClientEvents.OnAuthSuccess);
                    _state = State.Authenticated;
                    sendCommand();
                    break;
                default:
                    base.onClientEvent(ClientEvents.OnAuthFailure);
                    ServerCommands.QUITCommand quit = new ServerCommands.QUITCommand();
                    this.sendCommand(quit);
```

```
                    Shutdown();
                    break;
                }
            }
        }
    }
```

... and the client implementation.

ClientNetworkClient.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Net.Sockets;
using System.Diagnostics;
using Shared;

namespace Client
{
    /// <summary>
    /// A client implementation of aClient
    /// </summary>
    class ClientNetworkClient : aClient
    {
        /// <summary>
        /// Create a new network client
        /// </summary>
        /// <param name="client">The TcpClient to wrap around</param>
        /// <param name="hostname">The hostname</param>
        public ClientNetworkClient(TcpClient client, string hostname) : base(client,
hostname) { }

        /// <summary>
        /// Initialise SSL as a client
        /// </summary>
        /// <param name="ssl">The SSLStream to authenticate</param>
        /// <param name="hostname">The hostname</param>
        public override void initSSL(System.Net.Security.SslStream ssl, string
hostname)
        {
            Trace.TraceInformation("Authenticating as client for SSL.");
            ssl.AuthenticateAsClient(hostname, null,
System.Security.Authentication.SslProtocols.Tls, false);
        }

        /// <summary>
        /// Do nothing here - this is for server implementations
        /// </summary>
        public override void connectionInit()
        {
            //Do nothing on client
        }

        /// <summary>
        /// Process a new command
        /// </summary>
```

```
        /// <param name="messageText">The message text</param>
        /// <param name="sequenceNumber">The sequence number</param>
        public override void processNewCommand(string messageText, long sequenceNumber)
        {
            if (messageText == ProtocolText.BANNER)
            {
                ClientCommands.AUTHCommand AUTH = new
ClientCommands.AUTHCommand("hello");
                AUTH.SequenceNumber = sequenceNumber;
                sendCommand(AUTH);
                return;
            }

            if (messageText == ProtocolText.QUIT)
            {
                Trace.WriteLine("Received QUIT command from server.");
                Shutdown();
            }
        }

        /// <summary>
        /// Process a response from an aCommand object
        /// </summary>
        /// <param name="response">The response level</param>
        /// <param name="command">The aCommand that generated this response</param>
        public override void processResponse(Shared.Commands.Response response,
Shared.Commands.aCommand command)
        {
            switch (command.CommandType)
            {
                case Shared.Commands.CommandType.AUTH:
                    handleAuth(response, command);
                    break;
            }
        }

        /// <summary>
        /// Handle authentication
        /// </summary>
        /// <param name="response">The response level</param>
        /// <param name="command">The aCommand that generated this response</param>
        private void handleAuth(Shared.Commands.Response response,
Shared.Commands.aCommand command)
        {
            //AUTH check
            switch (response)
            {
                case Shared.Commands.Response.INFORMATION:
```

```
                    Trace.WriteLine(command.Information);
                    break;
                case Shared.Commands.Response.SUCCESS:
                    base.onClientEvent(ClientEvents.OnAuthSuccess);
                    _state = State.Authenticated;
                    break;
                default:
                    base.onClientEvent(ClientEvents.OnAuthFailure);
                    break;
            }
        }
    }
}
```

Now we need some ways to actually do stuff. In this example we therefore need Client and Server versions of the following commands: AUTH/BANNER and QUIT.

ServerAUTHCommand.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using Shared.Commands;
using System.Diagnostics;
using Shared;

namespace Server.ServerCommands
{
    /// <summary>
    /// A server banner implementation of aCommand
    /// </summary>
    class AUTHCommand : aCommand
    {
        string _bannerText = string.Empty;
        string _password = string.Empty;

        /// <summary>
        /// Create a new auth command
        /// </summary>
        /// <param name="password">The password that is expected</param>
        public AUTHCommand(string password)
        {
            Trace.TraceInformation("Created a server banner.");
            _bannerText = ProtocolText.BANNER;
            _password = password;

            setCommandText(new string[] { _bannerText });
        }

        #region aCommand Members

        /// <summary>
        /// The command type
        /// </summary>
        public override CommandType CommandType
        {
            get { return CommandType.BANNER; }
        }

        /// <summary>
        /// Method to be called internally when a response is processed
        /// </summary>
        /// <returns>A Response level</returns>
        public override Response ResponseDone()
        {
```

```
            if (ResponseText.Count == 0)
            {
                setCommandText(new string[] { ProtocolText.Positive });
                return Response.TERMINAL;
            }
            else if (ResponseText[0] != string.Format("{0} {1}",
ProtocolText.LOGINPrefix, _password))
            {
                setCommandText(new string[] { ProtocolText.Negative });
                return Response.TERMINAL;
            }

            setCommandText(new string[] { ProtocolText.Positive });
            return Response.SUCCESS;
        }

        #endregion
    }
}
```

ServerQUITCommand.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using Shared;
using Shared.Commands;
using System.Diagnostics;

namespace Server.ServerCommands
{
    /// <summary>
    /// A server QUIT command implementation of aCommand
    /// </summary>
    class QUITCommand : aCommand
    {
        #region aCommand Members

        /// <summary>
        /// Instantiate a new QUIT command
        /// </summary>
        public QUITCommand()
        {
            Trace.WriteLine("Created a server QUIT command.");
            this.setCommandText(new string[] { ProtocolText.QUIT });
        }

        /// <summary>
        /// The type of command
        /// </summary>
        public override CommandType CommandType
        {
            get { return CommandType.QUIT; }
        }

        /// <summary>
        /// The method called internally when a response is complete
        /// </summary>
        /// <returns>A response level indication whether the client
        /// accepted the proposed QUIT. They will be disconnected
        /// anyway.</returns>
        public override Response ResponseDone()
        {
            if (ResponseText[ResponseText.Count - 1] == ProtocolText.Positive)
            {
                return Response.SUCCESS;
            }
            else
```

```
            {
                return Response.ERROR;
            }
        }

        #endregion
    }
}
```

ClientAUTHCommand.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using Shared.Commands;
using System.Diagnostics;
using Shared;

namespace Client.ClientCommands
{
    /// <summary>
    /// A client AUTH implementation of aCommand
    /// </summary>
    class AUTHCommand: aCommand
    {
        /// <summary>
        /// The internal state of this AUTH command
        /// </summary>
        enum AUTHState
        {
            AUTH,
            Response
        }

        string _password = string.Empty;
        AUTHState _state = AUTHState.AUTH;

        #region aClientCommand Members

        /// <summary>
        /// The type of command
        /// </summary>
        public override CommandType CommandType
        {
            get { return CommandType.AUTH; }
        }

        /// <summary>
        /// The method called internally when a response is complete
        /// </summary>
        /// <returns>A Response level</returns>
        public override Response ResponseDone()
        {
            if (_state == AUTHState.AUTH)
            {
                _state = AUTHState.Response;
                setInformation("Client was in the AUTH phase and as yet has no
```

```
response.");
                return Response.INFORMATION;
            }

            if (ResponseText[ResponseText.Count - 1] == ProtocolText.Positive)
            {
                return Response.SUCCESS;
            }
            else
            {
                return Response.TERMINAL;
            }
        }

        #endregion

        /// <summary>
        /// Instantiates a new client AUTH command object
        /// </summary>
        /// <param name="password">The password to login with</param>
        public AUTHCommand(string password)
        {
            Trace.TraceInformation("Created a client AUTH response.");
            _password = password;
            this.setCommandText(new string[] { string.Format("{0} {1}",
ProtocolText.LOGINPrefix, _password) });
        }
    }
}
```

Right, now the final two components - server and client wrappers to create those pesky TcpClients!

ClientWrapper.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Net.Sockets;
using Shared;

namespace Client
{
    /// <summary>
    /// A class to wrap a TcpClient inside an aClient object
    /// </summary>
    class ClientWrapper
    {
        /// <summary>
        /// A callback for ClientConnectAttempt
        /// </summary>
        /// <param name="client">The aClient object or null if it failed to
connect</param>
        public delegate void ClientConnectAttempt(aClient client);

        /// <summary>
        /// An event that is raised when a client finishes a connection attempt
        /// </summary>
        public event ClientConnectAttempt OnClientConnectAttemptComplete;

        string _host = string.Empty;
        int _port = 0;
        TcpClient _client = null;

        /// <summary>
        /// Instantiate a new ClientWrapper
        /// </summary>
        /// <param name="host">The host to connect to</param>
        /// <param name="port">The port to connect to</param>
        public ClientWrapper(string host, int port)
        {
            _host = host;
            _port = port;

            _client = new TcpClient();
            _client.BeginConnect(host, port, new
AsyncCallback(beginConnectTcpClientCallback), _client);
        }

        /// <summary>
        /// A callback for the TcpClient's connection attempt
```

```
        /// </summary>
        /// <param name="ar">An IAsyncResult which has the TcpClient as its
        /// AsyncState property</param>
        private void beginConnectTcpClientCallback(IAsyncResult ar)
        {
            TcpClient client = (TcpClient)ar.AsyncState;

            lock (this)
            {
                aClient cnc = null;

                if (client.Connected)
                {
                    cnc = new ClientNetworkClient(client, _host);
                }

                if (OnClientConnectAttemptComplete != null)
                {
                    OnClientConnectAttemptComplete(cnc);
                }
            }
        }
    }
}
```

Server.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Net.Sockets;
using System.Threading;
using System.Net.Security;
using System.Security.Cryptography.X509Certificates;
using Shared;
using System.Diagnostics;

namespace Server
{
    /// <summary>
    /// A server wrapper class
    /// </summary>
    class Server
    {
        ManualResetEvent _tcpClientConnected = new ManualResetEvent(false);
        TcpListener _listener = null;
        bool _shutdown = false;
        List<aClient> _clients = new List<aClient>();

        /// <summary>
        /// Instantiate a new server
        /// </summary>
        /// <param name="port">The port to listen on</param>
        public Server(int port)
        {
            _listener = new TcpListener(System.Net.IPAddress.Any, port);
            _listener.Start();

            while (!_shutdown)
            {
                DoBeginAcceptTcpClient(_listener);
            }

            _listener.Stop();
        }

        /// <summary>
        /// Shutdown this server gracefully
        /// </summary>
        public void Shutdown()
        {
            _shutdown = true;
            _tcpClientConnected.Set();
```

```
        }

        /// <summary>
        /// Begin accepting a socket
        /// </summary>
        /// <param name="listener">The TcpListener</param>
        private void DoBeginAcceptTcpClient(TcpListener
            listener)
        {
            // Set the event to nonsignaled state.
            _tcpClientConnected.Reset();

            // Start to listen for connections from a client.
            System.Diagnostics.Trace.TraceInformation("Waiting for a connection...");

            // Accept the connection.
            // BeginAcceptSocket() creates the accepted socket.
            listener.BeginAcceptTcpClient(
                new AsyncCallback(DoAcceptTcpClientCallback),
                listener);

            // Wait until a connection is made and processed before
            // continuing.
            _tcpClientConnected.WaitOne();
        }

        /// <summary>
        /// Process a client connection
        /// </summary>
        /// <param name="ar">An ar with a TcpListener as its AsyncState
        /// property</param>
        private void DoAcceptTcpClientCallback(IAsyncResult ar)
        {
            // Get the listener that handles the client request.
            TcpListener listener = (TcpListener)ar.AsyncState;

            // End the operation and display the received data on
            // the console.
            TcpClient client = listener.EndAcceptTcpClient(ar);

            // Process the connection here. (Add the client to a
            // server table, read data, etc.)
            System.Diagnostics.Trace.TraceInformation("Client connected completed");

            ServerNetworkClient nc = new ServerNetworkClient(client);
            nc.OnAuthFailure += new aClient.ClientEvent(nc_OnAuthFailure);
            nc.OnAuthSuccess += new aClient.ClientEvent(nc_OnAuthSuccess);
```

```
            _clients.Add(nc);

            //Start the client (non-blocking)
            nc.Start();

            // Signal the calling thread to continue.
            _tcpClientConnected.Set();

        }

        /// <summary>
        /// A callback for successful logins
        /// </summary>
        /// <param name="client">The aClient that raised the event</param>
        void nc_OnAuthSuccess(aClient client)
        {
            Trace.TraceInformation("Succesfully logged in.");
        }

        /// <summary>
        /// A callback for failed logins
        /// </summary>
        /// <param name="client">The aClient that raised the event</param>
        void nc_OnAuthFailure(aClient client)
        {
            Trace.TraceInformation("Login failure.");
            _clients.Remove(client);
        }

    }
}
```

So, the final things you need to get up and running are examples of a client and server Program.cs to actually start the processes.

ClientProgram.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using Shared;

namespace Client
{
    class Program
    {
        static aClient _client = null;

        static void Main(string[] args)
        {
            System.Diagnostics.Trace.TraceInformation("Client");
            Console.WriteLine("Sleeping for 2 seconds to let server start.");
            System.Threading.Thread.Sleep(2000);

            ClientWrapper cw = new ClientWrapper("localhost", 9000);
            cw.OnClientConnectAttemptComplete += new
ClientWrapper.ClientConnectAttempt(cw_OnClientConnectAttemptComplete);

            Console.WriteLine("Running. Press any key to end.");
            Console.ReadKey();
        }

        static void cw_OnClientConnectAttemptComplete(aClient client)
        {
            if (client != null)
            {
                client.OnAuthFailure += new aClient.ClientEvent(c_OnAuthFailure);
                client.OnAuthSuccess += new aClient.ClientEvent(c_OnAuthSuccess);

                client.Start();

                _client = client;
            }
            else
            {
                Console.WriteLine("Client timed out.");
            }
        }

        static void c_OnAuthSuccess(aClient client)
        {
            System.Diagnostics.Trace.TraceInformation("Client logged in.");
```

```
            System.Threading.Thread.Sleep(1000);

            client.Shutdown();
        }

        static void c_OnAuthFailure(aClient client)
        {
            System.Diagnostics.Trace.TraceInformation("Client login failure.");
        }
    }
}
```

ServerProgram.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.IO;
using System.Windows.Forms;

namespace Server
{
    class Program
    {
        static void Main(string[] args)
        {

            Console.WriteLine("Server has started.");
            Server s = new Server(9000);
            Console.WriteLine("Server has ended.");
            Console.ReadKey();
            return;
        }
    }
}
```

And there you have it - CRC verified, SSL enabled communications! My next update is for binary data transfer.